

XCPC 算法模板

usedchang

2026 年 3 月 13 日

目录

1	杂项	3
1.1	IO 加速	3
1.2	对拍	4
2	数据结构	5
2.1	ST 表	5
2.2	并查集	6
2.3	线段树 (普通区间和查询)	9
2.4	笛卡尔树	10
2.5	字典树	11
2.6	莫队	14
3	树	15
3.1	LCA	15
3.2	树的直径	17
3.3	树链剖分	19
4	图论	25
4.1	SPFA	25
4.2	Dijkstra	26
4.3	最小生成树	29
4.4	Kruskal 重构树	30
4.5	差分约束	32
4.6	缩点	34
5	字符串	35
5.1	字符串哈希	35

5.2	KMP	35
5.3	后缀树	35
5.4	马拉车	35
6	动态规划	36
6.1	01 背包	36
6.2	完全背包	36
6.3	多重背包	36
6.4	树形 dp	36
6.5	换根 dp	36
6.6	状压 dp	36
6.7	SOS dp	36
6.8	数位 dp	36
7	数学	37
7.1	筛法求因数个数	37
7.2	筛法求欧拉函数	37
7.3	筛法求莫比乌斯函数	39
7.4	矩阵快速幂	40
7.5	第二类斯特林数	42
7.6	卡特兰数	43
7.7	排列组合类-含 Lucas	43
7.8	高斯消元	43
7.9	线性基	43
7.10	数论分块	43
7.11	莫比乌斯反演	43
7.12	exgcd	43
7.13	位运算基础	43

1 杂项

1.1 IO 加速

Listing 1: 快速读入模板

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // ===== 通用快读（支持任意整数类型） =====
5  template<typename T>
6  inline T read() {
7      T x = 0; int f = 1; char ch = getchar();
8      while (ch < '0' || ch > '9') { if (ch == '-') f = -1; ch = getchar(); }
9      while (ch >= '0' && ch <= '9') { x = x * 10 + (ch - '0'); ch = getchar(); }
10     return x * f;
11 }
12
13 // ===== 通用快写（支持任意整数类型） =====
14 template<typename T>
15 inline void write(T x) {
16     if (x < 0) { putchar('-'); x = -x; }
17     if (x > 9) write(x / 10);
18     putchar(x % 10 + '0');
19 }
20
21 // ===== 使用示例 =====
22 int main() {
23     int a = read<int>();
24     long long b = read<long long>();
25     short c = read<short>();
26     __int128 d = read<__int128>(); // 支持 __int128!
27
28     write(a); putchar('\n');
29     write(b); putchar(' ');
30     write(c); putchar('\n');
31     write(d); putchar('\n'); // __int128 直接输出完美
32     return 0;
33 }
```

1.2 对拍

Listing 2: 对拍 windows

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 int main(){
4     //一直循环，直到找到不一样的数据
5     while (1) {
6         system("data.exe > data.txt");
7         system("bf.exe < data.txt>bf.txt");
8         system("test.exe < data.txt > test.txt");
9         if (system("fc test.txt bf.txt")) break;
10        //不一样就跳出循环
11    }//当 fc 返回 1 时，说明这时数据不一样
12    return 0;
13 }
```

Listing 3: 对拍 linux

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 int main(){
4     //一直循环，直到找到不一样的数据
5     while (1) {
6         system("./data > data.txt");
7         system("./bf < data.txt > bf.txt");
8         system("./test < data.txt > test.txt");
9         if (system("diff test.txt bf.txt")) break;
10        //不一样就跳出循环
11    }//当 diff 返回非0 时，说明这时数据不一样
12    return 0;
13 }
```

2 数据结构

2.1 ST 表

Listing 4: ST 表

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  #define endl '\n'
4  typedef long long ll;
5  const ll INF=4e18;
6  struct ST{
7      ST(){};
8      vector<vector<ll>>mx,mn;
9      ST(const vector<ll>&a){
10         int n=a.size()-1;
11         mx=vector<vector<ll>>(n+1,vector<ll>(21,-INF));
12         mn=vector<vector<ll>>(n+1,vector<ll>(21,INF));
13         for(int i=1;i<=n;i++) mx[i][0]=mn[i][0]=a[i];
14         for(int i=1;i<=21;i++){
15             for(int j=1;j+(1<<i)-1<=n;j++){
16                 mx[j][i]=(mx[j][i-1]>mx[j+(1<<i-1)][i-1])?mx[j][i-1]:mx[j+(1<<i-1)][i-1];
17                 mn[j][i]=(mn[j][i-1]<mn[j+(1<<i-1)][i-1])?mn[j][i-1]:mn[j+(1<<i-1)][i-1];
18             }
19         }
20     };
21     ll qry_mx(int l,int r){
22         int len=__lg(r-l+1);//31-__builtin_clz(r-l+1)
23         return (mx[l][len]>mx[r-(1<<len)+1][len])?mx[l][len]:mx[r-(1<<len)+1][len];
24     }
25     ll qry_mn(int l,int r){
26         int len=__lg(r-l+1);//31-__builtin_clz(r-l+1)
27         return (mn[l][len]<mn[r-(1<<len)+1][len])?mn[l][len]:mn[r-(1<<len)+1][len];
28     }
29 };
30 int main(){
31     cin.tie(0)->ios::sync_with_stdio(false);

```

```
32     int n,q,l,r;cin>>n>>q;
33     vector<ll>a(n+1);
34     for(int i=1;i<=n;i++) cin>>a[i];
35     ST st(a);
36     while(q--){
37         cin>>l>>r;
38         cout<<st.qry_mx(l,r)<<endl;
39     }
40     return 0;
41 }
```

2.2 并查集

Listing 5: 普通并查集

```
1 struct DSU{
2     vector<int>f;
3     vector<int>siz;
4     DSU(){};
5     DSU(int n){
6         f.resize(n+1);
7         iota(f.begin(),f.end(),0);
8         siz.assign(n+1,1);
9     }
10    int find(int x){
11        return x==f[x]?x:f[x]=find(f[x]);
12    }
13    bool merge(int x,int y){
14        int fx=find(x),fy=find(y);
15        if(fx==fy) return false;
16        f[fx]=fy;
17        siz[fy]+=siz[fx];
18        return true;
19    }
20    bool same(int x,int y){
21        return find(x)==find(y);
22    }
23 };
```

Listing 6: weighted-dsu

```
1 //P1196银河英雄传说
2 struct DSU{
3     DSU(){}
4     vector<int>f,siz;
5     vector<ll>dis;
6     DSU(int n){
7         dis.resize(n+1);
8         f.resize(n+1);
9         iota(f.begin(),f.end(),0);
10        siz.assign(n+1,1);
11    }
12    int find(int x){
13        int fx=f[x];
14        if(fx==x) return x;
15        int rt=find(f[x]);
16        dis[x]+=dis[f[x]];
17        return f[x]=rt;
18    }
19    bool merge(int x,int y){
20        int fx=find(x),fy=find(y);
21        if(fx==fy) return false;
22        f[fx]=fy;
23        dis[fx]+=siz[fy];
24        siz[fy]+=siz[fx];
25        return true;
26    }
27    bool same(int x,int y){
28        return find(x)==find(y);
29    }
30    ll Dis(int x,int y){
31        return dis[x]-dis[y];
32    }
33 };
34 void solve(){
35     int n;cin>>n;
36     DSU dsu(n+1);
37     for(int i=1;i<=n;i++){
38         char c;int x,y;cin>>c>>x>>y;
39         if(c=='M') dsu.merge(x,y);
```

```
40         else{
41             if(dsu.same(x,y)) cout<<abs(dsu.Dis(x,y))-1<<endl;
42             else cout<<-1<<endl;
43         }
44     }
45 }
```


2.3 线段树 (普通区间和查询)

Listing 7: common-Seg

```

1 struct Seg{
2     #define lc (p<<1)
3     #define rc (p<<1|1)
4     struct S{
5         int l,r;
6         ll val,laz;
7     };
8     Seg(){};
9     Seg(int n){
10         tre.resize(n<<2);
11         build(0,n,1);
12     };
13     void pushup(int p){
14         tre[p].val=tre[lc].val+tre[rc].val;
15     }
16     void pushdown(int p){
17         if(tre[p].laz==0) return;
18         tre[lc].laz+=tre[p].laz;
19         tre[rc].laz+=tre[p].laz;
20         tre[lc].val+=tre[p].laz;
21         tre[rc].val+=tre[p].laz;
22         tre[p].laz=0;
23     }
24     void build(int l,int r,int p){
25         tre[p]={l,r,0,0};
26         if(l==r){
27             tre[p]=a[l];
28             return;
29         }
30         int mid=l+r>>1;
31         build(l,mid,lc);
32         build(mid+1,r,rc);
33         pushup(p);
34     }
35     void add(int l,int r,ll k,int p){
36         if(l<=tre[p].l&&tr>=tre[p].r){
37             tre[p].val+=k*(tre[p].r-tre[p].l+1);

```

```

38         tre[p].laz+=k;
39         return;
40     }
41     int mid=tre[p].l+tre[p].r>>1;
42     pushdown(p);
43     if(l<=mid) add(l,r,k,lc);
44     if(r>mid) add(l,r,k,rc);
45     pushup(p);
46 }
47 ll qry(int l,int r,int p){
48     if(l<=tre[p].l&& r>=tre[p].r){
49         return tre[p].val;
50     }
51     int mid=tre[p].l+tre[p].r>>1;
52     pushdown(p);
53     ll ans=0;
54     if(l<=mid) ans+=qry(l,r,lc);
55     if(r>mid) ans+=qry(l,r,rc);
56     return ans;
57 }
58 };

```

2.4 笛卡尔树

Listing 8: cartesian-tree

```

1  //P7972 Self Permutation
2  #include<bits/stdc++.h>
3  using namespace std;
4  #define endl '\n'
5  typedef long long ll;
6  const ll mod=1e9+7;
7  void solve(){
8      int n;cin>>n;
9      vector<int>L(n+1),R(n+1),st,a(n+1);
10     vector<ll>dp(n+1);
11     for(int i=1;i<=n;i++){
12         cin>>a[i];
13         int lc=0;
14         while(!st.empty()&&a[st.back()]>=a[i]){

```

```

15         lc=st.back();
16         st.pop_back();
17     }
18     L[i]=lc;
19     if(!st.empty()) R[st.back()]=i;
20     st.emplace_back(i);
21 }
22 auto dfs=[&](auto &&dfs,int u,int l,int r) ->void {
23     if(L[u]) dfs(dfs,L[u],l,u-1);
24     if(R[u]) dfs(dfs,R[u],u+1,r);
25     (dp[u]+=(dp[L[u]]+1)*(dp[R[u]]+1)%mod)%=mod;
26     if(l>1) (dp[u]+=dp[R[u]])%=mod;
27     if(r<n) (dp[u]+=dp[L[u]])%=mod;
28 };
29 dfs(dfs,st.front(),1,n);
30 cout<<dp[st.front()]<<endl;
31 }

```

2.5 字典树

Listing 9: Trie

```

1 struct Trie{
2     vector<int>cnt;
3     vector< vector<int> >son;
4     int idx=0;
5     Trie(int n){
6         cnt.resize(n+1);
7         son.resize(n+1);
8         for(int i=0;i<=n;i++) son[i]=vector<int>(65);
9     }
10    void insert(string &s){
11        int num;
12        int p=0;
13        for(char &c:s){
14            if(isdigit(c)) num=c-'0'+52;
15            else if(islower(c)) num=c-'a'+26;
16            else if(isupper(c)) num=c-'A';
17            if(!son[p][num]) son[p][num]=(++idx);
18            p=son[p][num];

```

```

19         ++cnt[p];
20     }
21 }
22 int query(string &s){
23     int num;
24     int p=0;
25     for(char &c:s){
26         if(isdigit(c)) num=c-'0'+52;
27         else if(islower(c)) num=c-'a'+26;
28         else if(isupper(c)) num=c-'A';
29         if(!son[p][num]) return 0;
30         p=son[p][num];
31     }
32     return cnt[p];
33 }
34 }trie;

```

Listing 10: 01-Trie

```

1 //P4551 最长异或路径
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define endl '\n'
5 typedef long long ll;
6 struct ZTrie{
7     vector<array<int,2>>son;
8     vector<int>cnt;
9     int idx=0;
10    ZTrie(int n){
11        son.resize(n+1);
12        cnt.resize(n+1);
13    }
14    void insert(int x){
15        int p=0;
16        for(int i=31;i>=0;i--) {
17            int bit=(x>>i&1);
18            if(!son[p][bit]) son[p][bit]=(++idx);
19            p=son[p][bit];
20        }
21    }
22    int query(int x){

```

```
23     int ans=0,p=0;
24     for(int i=31;i>=0;i--){
25         int bit=(x>>i&1);
26         if(son[p][bit^1]) p=son[p][bit^1],ans|=(1<<i);
27         else if(son[p][bit]) p=son[p][bit];
28         else return ans;
29     }
30     return ans;
31 }
32 }trie(2e6);
33 const int N=2e5+5;
34 struct node{
35     int u,w;
36 };
37 vector<node>G[N];
38 int ans=0;
39 void dfs(int u,int fa,int sum){
40     trie.insert(sum);
41     ans=max(ans,trie.query(sum));
42     for(auto &[v,w]:G[u]){
43         if(v==fa) continue;
44         dfs(v,u,sum^w);
45     }
46 }
47 void solve(){
48     int n;cin>>n;
49     for(int i=1;i<n;i++){
50         int x,y,w;cin>>x>>y>>w;
51         G[x].push_back({y,w});
52         G[y].push_back({x,w});
53     }
54     dfs(1,0,0);
55     cout<<ans<<endl;
56 }
57 int main(){
58     cin.tie(0)->ios::sync_with_stdio(false);
59     solve();
60     return 0;
61 }
```

2.6 莫队

Listing 11: 普通莫队算法

```

1 //P2709 小B的询问
2 const int N=5e4+10;
3 int a[N],pos[N],cnt[N];
4 ll res,ans[N];
5 struct Q{
6     int l,r,k;
7 }q[N];
8 void Add(int n){
9     cnt[a[n]]++;
10    res+=cnt[a[n]]*cnt[a[n]]-(cnt[a[n]]-1)*(cnt[a[n]]-1);
11 }
12 void Sub(int n){
13     cnt[a[n]]--;
14     res-=(cnt[a[n]]+1)*(cnt[a[n]]+1)-(cnt[a[n]]*(cnt[a[n]]));
15 }
16 int main(){
17     ios::sync_with_stdio(false);
18     cin.tie(0),cout.tie(0);
19     int n,m,k;cin>>n>>m>>k;
20     int siz=sqrt(n+0.5);
21     for(int i=1;i<=n;i++){
22         cin>>a[i];
23         pos[i]=i/siz;
24     }
25     for(int i=0;i<m;i++){
26         cin>>q[i].l>>q[i].r;
27         q[i].k=i;
28     }
29     sort(q,q+m,[](Q x,Q y){
30         return pos[x.l]==pos[y.l]?x.r<y.r:pos[x.l]<pos[y.l];
31     });
32     int l=1,r=0;
33     for(int i=0;i<m;i++){
34         while(q[i].l<l) Add(--l);
35         while(q[i].r>r) Add(++r);
36         while(q[i].l>l) Sub(l++);
37         while(q[i].r<r) Sub(r--);

```

```
38     ans[q[i].k]=res;
39 }
40 for(int i=0;i<m;i++){
41     cout<<ans[i]<<endl;
42 }
43 return 0;
44 }
```

3 树

3.1 LCA

Listing 12: dfn 大跳 LCA

```
1 //P3379 最近公共祖先
2 const int N=5e5+5;
3 vector<int>G[N];
4 int f[N],d[N],dfn[N],top[N],cnt,siz[N],son[N];
5 void dfs1(int u,int fa){
6     d[u]=d[fa]+1;
7     f[u]=fa;
8     siz[u]=1;
9     for(int v:G[u]){
10         if(fa==v) continue;
11         dfs1(v,u);
12         siz[u]+=siz[v];
13         if(siz[v]>siz[son[u]]) son[u]=v;
14     }
15 }
16 void dfs2(int u,int node){
17     dfn[u]=(++cnt);//维护dfn序
18     top[u]=node;
19     if(son[u]) dfs2(son[u],node);
20     for(int v:G[u]){
21         if(v==f[u]||v==son[u]) continue;
22         dfs2(v,v);
23     }
24 }
25 int LCA(int l,int r){
```

```

26     while(top[l]!=top[r]){
27         if(d[top[l]]<d[top[r]]) swap(l,r);
28         l=f[top[l]];
29     }
30     if(d[l]<d[r]) return l;
31     return r;
32 }//树上LCA跳
33 void solve(){
34     int n,m,s;
35     cin>>n>>m>>s;
36     for(int i=1;i<n;i++){
37         int x,y;cin>>x>>y;
38         G[x].push_back(y);
39         G[y].push_back(x);
40     }
41     dfs1(s,0);
42     dfs2(s,s);
43     for(int i=1;i<=m;i++){
44         int x,y;cin>>x>>y;
45         cout<<LCA(x,y)<<endl;
46     }
47 }

```

Listing 13: dfn 实现 O1 LCA

```

1 //P3379 最近公共祖先
2 void solve(){
3     int n,q,s;
4     cin>>n>>q>>s;
5     vector<vector<int>>>G(n+1);
6     for(int i=1;i<n;i++){
7         int x,y;
8         cin>>x>>y;
9         G[x].push_back(y);
10        G[y].push_back(x);
11    }
12    vector<int>d(n+1);
13    vector<int>f(n+1);
14    vector<int>dfn(n+1);
15    int cnt=0;
16    auto dfs=[&](auto &&dfs,int u,int fa) ->void {

```



```

17     f[u]=fa;
18     dfn[u]=(++cnt);
19     d[u]=d[fa]+1;
20     for(int v:G[u]){
21         if(v==fa) continue;
22         dfs(dfs,v,u);
23     }
24 };
25 dfs(dfs,s,0);
26 vector<vector<int>>st(n+1,vector<int>(21));
27 for(int i=1;i<=n;i++) st[dfn[i]][0]=f[i];
28 for(int j=1;j<=20;j++){
29     for(int i=1;i+(1<<j)-1<=n;i++){
30         int l=st[i][j-1];
31         int r=st[i+(1<<j-1)][j-1];
32         if(d[l]<d[r]) st[i][j]=l;
33         else st[i][j]=r;
34     }
35 }
36 auto lca=[&](int l,int r) ->int {
37     if(l==r) return l;
38     l=dfn[l],r=dfn[r];
39     if(l>r) swap(l,r);
40     l++;
41     int len=__lg(r-l+1);
42     int L=st[l][len];
43     int R=st[r-(1<<len)+1][len];
44     return (d[L]<d[R])?L:R;
45 };
46 while(q--){
47     int x,y;
48     cin>>x>>y;
49     cout<<lca(x,y)<<endl;
50 }
51 }

```

3.2 树的直径

Listing 14: 两遍 dfs 查直径

```

1  const int N=2e5+10;
2  vector<int>G[N];
3  bool vis[N];
4  ll d[N],s=1;
5  void dfs(int u,int f){
6      for(auto &v:G[u]){
7          if(v==f) continue;
8          d[v]=d[u]+1;
9          dfs(v,u);
10     }
11 }
12 int main(){
13     cin.tie(0)->ios::sync_with_stdio(false);
14     int n;cin>>n;
15     for(int i=1,x,y;i<n;i++){
16         cin>>x>>y;
17         G[x].push_back(y);
18         G[y].push_back(x);
19     }
20     d[s]=0;dfs(s,0);
21     for(int i=1;i<=n;i++) if(d[i]>d[s]) s=i;
22     d[s]=0;dfs(s,0);
23     ll ans=0;
24     for(int i=1;i<=n;i++) ans=max(ans,d[i]);
25     cout<<ans<<endl;
26     return 0;
27 }

```

Listing 15: 树形 dp 求直径

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  #define endl '\n'
4  typedef long long ll;
5  int main(){
6     cin.tie(0)->ios::sync_with_stdio(false);
7     int n;cin>>n;
8     vector<vector<int>>G(n+1);
9     vector<int>down(n+1),up(n+1);
10    for(int i=1;i<n;i++){
11        int x,y;cin>>x>>y;

```

```

12     G[x].emplace_back(y);
13     G[y].emplace_back(x);
14 }
15 auto dfs1=[&](auto &&dfs1,int u,int fa) ->void {
16     for(int v:G[u]){
17         if(v==fa) continue;
18         dfs1(dfs1,v,u);
19         down[u]=max(down[u],down[v]+1);
20     }
21 };
22 dfs1(dfs1,1,0);
23 auto dfs2=[&](auto &&dfs2,int u,int fa) ->void {
24     int m1=0,m2=0;//最长路径,次长路径
25     for(int v:G[u]){
26         if(v==fa) continue;
27         if(down[m1]<down[v]) m2=m1,m1=v;
28         else if(down[m2]<down[v]) m2=v;
29     }
30     for(int v:G[u]){
31         if(v==fa) continue;
32         up[v]=up[u]+1;
33         if(m1!=0&&down[m1]!=down[v]) up[v]=max(up[v],down[m1]+2);
34         else if(down[m2]!=0) up[v]=max(up[v],down[m2]+2);
35         dfs2(dfs2,v,u);
36     }
37 };
38 dfs2(dfs2,1,0);
39 int ans=0;
40 for(int i=1;i<=n;i++) ans=max(ans,down[i]+up[i]);
41 cout<<ans<<endl;
42 return 0;
43 }

```

3.3 树链剖分

Listing 16: 树链剖分

```

1 //P1505 国家集训队
2 const int N=2e5+5;
3 int f[N],d[N],siz[N],dfn[N],son[N],top[N],cnt;

```

```
4 ll a[N],b[N];
5 const ll INF=1e18;
6 struct Edge{
7     int x,y;ll w;
8 };
9 vector<int>G[N];
10 void dfs1(int u,int fa){
11     d[u]=d[fa]+1;
12     f[u]=fa;
13     siz[u]=1;
14     for(int v:G[u]){
15         if(v==fa) continue;
16         dfs1(v,u);
17         siz[u]+=siz[v];
18         if(siz[v]>siz[son[u]]) son[u]=v;
19     }
20 }
21 void dfs2(int u,int node){
22     top[u]=node;
23     dfn[u]=(++cnt);
24     b[cnt]=a[u];
25     if(son[u]){
26         dfs2(son[u],node);
27     }
28     for(int v:G[u]){
29         if(v==f[u] || v==son[u]) continue;
30         dfs2(v,v);
31     }
32 }
33 struct Info{
34     ll sum,mx,mn;
35     Info(ll sum,ll mx,ll mn):sum(sum),mx(mx),mn(mn){};
36     Info():sum(0LL),mx(-INF),mn(INF){};
37     friend Info operator+(const Info A,const Info B){
38         return Info(A.sum+B.sum,max(A.mx,B.mx),min(A.mn,B.mn));
39     }
40     friend Info operator+=(Info &A,const Info B){
41         A=A+B;
42         return A;
43     }
```

```

44 };
45 struct Seg{
46     struct S{
47         int l,r;
48         ll sum,mx,mn,laz;bool laz1;
49     };
50     #define lc (p<<1)
51     #define rc (p<<1|1)
52     vector<S>tre;
53     Seg(){};
54     Seg(int n){
55         tre.resize(n<<2);
56         build(1,n,1);
57     }
58     void pushdown(int p){
59         if(tre[p].laz!=INF) {
60             tre[lc].laz=tre[rc].laz=tre[p].laz;
61             tre[lc].laz1=tre[rc].laz1=false;
62             tre[lc].mx=tre[rc].mx=tre[p].laz;
63             tre[lc].mn=tre[rc].mn=tre[p].laz;
64             tre[lc].sum=tre[p].laz*(tre[lc].r-tre[lc].l+1);
65             tre[rc].sum=tre[p].laz*(tre[rc].r-tre[rc].l+1);
66             tre[p].laz=INF;
67         }
68         if(tre[p].laz1){
69             tre[lc].laz1^=tre[p].laz1;
70             tre[rc].laz1^=tre[p].laz1;
71
72             tre[lc].sum=-tre[lc].sum;
73             ll lasmn=tre[lc].mn,lasmx=tre[lc].mx;
74             tre[lc].mx=-lasmn;
75             tre[lc].mn=-lasmx;
76
77             tre[rc].sum=-tre[rc].sum;
78             lasmn=tre[rc].mn,lasmx=tre[rc].mx;
79             tre[rc].mx=-lasmn;
80             tre[rc].mn=-lasmx;
81             tre[p].laz1=0;
82         }
83     }

```

```
84 void pushup(int p){
85     tre[p].mx=max(tre[lc].mx,tre[rc].mx);
86     tre[p].mn=min(tre[lc].mn,tre[rc].mn);
87     tre[p].sum=tre[lc].sum+tre[rc].sum;
88 }
89 void build(int l,int r,int p){
90     tre[p]={l,r,0,-INF,INF,INF,0};
91     if(l==r){
92         tre[p].sum=b[l];
93         tre[p].mx=tre[p].mn=b[l];
94         return;
95     }
96     int mid=l+r>>1;
97     build(l,mid,lc);
98     build(mid+1,r,rc);
99     pushup(p);
100 }
101 void Change(int l,int r,ll k,int p){
102     if(l<=tre[p].l&&tr>=tre[p].r){
103         tre[p].laz1=false;
104         tre[p].laz=tre[p].mx=tre[p].mn=k;
105         tre[p].sum=k*(tre[p].r-tre[p].l+1);
106         return;
107     }
108     int mid=tre[p].l+tre[p].r>>1;
109     pushdown(p);
110     if(l<=mid) Change(l,r,k,lc);
111     if(r>mid) Change(l,r,k,rc);
112     pushup(p);
113 }//区间赋值
114 void Flip(int l,int r,int p){
115     if(l<=tre[p].l&&tr>=tre[p].r){
116         tre[p].laz1^=1;
117         tre[p].sum*=-1;
118         ll lasmn=tre[p].mn,lasmx=tre[p].mx;
119         tre[p].mx=-lasmn;
120         tre[p].mn=-lasmx;
121         return;
122     }
123     int mid=tre[p].l+tre[p].r>>1;
```

```

124     pushdown(p);
125     if(l<=mid) Flip(l,r,lc);
126     if(r>mid) Flip(l,r,rc);
127     pushup(p);
128 }//区间乘-1
129 Info qry(int l,int r,int p){
130     if(l<=tre[p].l&&tre[p].r<=r){
131         return Info(tre[p].sum,tre[p].mx,tre[p].mn);
132     }
133     int mid=tre[p].l+tre[p].r>>1;
134     Info ans;
135     pushdown(p);
136     if(l<=mid) ans+=qry(l,r,lc);
137     if(r>mid) ans+=qry(l,r,rc);
138     return ans;
139 }
140 }S;
141 void Flip(int x,int y){
142     while(top[x]!=top[y]){
143         if(d[top[x]]<d[top[y]]) swap(x,y);
144         S.Flip(dfn[top[x]],dfn[x],1);
145         x=f[top[x]];
146     }
147     if(dfn[x]>dfn[y]) swap(x,y);
148     S.Flip(dfn[x]+1,dfn[y],1);
149 }
150 Info Qry(int x,int y){
151     Info T;
152     while(top[x]!=top[y]){
153         if(d[top[x]]<d[top[y]]) swap(x,y);
154         T+=S.qry(dfn[top[x]],dfn[x],1);
155         x=f[top[x]];
156     }
157     if(dfn[x]>dfn[y]) swap(x,y);
158     T+=S.qry(dfn[x]+1,dfn[y],1);
159     return T;
160 }
161 void solve(){
162     int n;cin>>n;
163     vector<Edge>edge;

```

```
164     for(int i=1;i<n;i++){
165         int x,y;ll w;cin>>x>>y>>w;
166         ++x,++y;
167         G[x].push_back(y);
168         G[y].push_back(x);
169         edge.push_back({x,y,w});
170     }
171     dfs1(1,0);
172     vector<int>mp(n+1);//输入的第i条边权下放到哪个点?
173     for(int i=0;i<edge.size();i++){
174         auto &[x,y,w]=edge[i];
175         if(d[x]>d[y]) {
176             a[x]=w;
177             mp[i+1]=x;
178         }
179         else {
180             a[y]=w;
181             mp[i+1]=y;
182         }
183     }
184     dfs2(1,1);
185     S=Seg(n);
186     int q;cin>>q;
187     while(q--){
188         string op;cin>>op;
189         int x,y;cin>>x>>y;
190         if(op=="SUM") {
191             ++x,++y;
192             cout<<Qry(x,y).sum<<endl;
193             //询问 x,y 节点之间边权和
194         }
195         else if(op=="MAX") {
196             ++x,++y;
197             cout<<Qry(x,y).mx<<endl;
198             //询问 x,y 节点之间边权最大值
199         }
200         else if(op=="MIN"){
201             ++x,++y;
202             cout<<Qry(x,y).mn<<endl;
203             //询问 x,y 节点之间边权最小值
```



```

204     }
205     else if(op=="C"){
206         x=mp[x];
207         S.Change(dfn[x],dfn[x],y,1);
208         //将输入的第 x 条边权值改为 y
209     }
210     else if(op=="N"){
211         ++x,++y;
212         Flip(x,y);
213         //将 u,v 节点之间的边权都变为相反数
214     }
215 }
216 }

```

4 图论

4.1 SPFA

Listing 17: SPFA

```

1 struct node{
2     int u,ll w;
3 };
4 vector<node>G[N];
5 ll dis[N];
6 bool inq[N];
7 int n,m,s;
8 void spfa(int s){
9     fill(dis+1,dis+1+n,INT_MAX);
10    queue<int>Q;
11    Q.push(s);
12    dis[s]=0;
13    inq[s]=1;
14    while(!Q.empty()){
15        int u=Q.front();
16        Q.pop();
17        inq[u]=0;
18        for(auto &[v,w]:G[u]){
19            if(dis[v]>dis[u]+w){

```

```

20         dis[v]=dis[u]+w;
21         if(!inq[v]){
22             Q.push(v);
23             inq[v]=1;
24         }
25     }
26 }
27 }
28 }
29 void solve(){
30     cin>>n>>m>>s;
31     for(int i=1;i<=m;i++){
32         int x,y;ll w;
33         cin>>x>>y>>w;
34         G[x].push_back({y,w});
35     }
36     spfa(s);
37     for(int i=1;i<=n;i++) cout<<dis[i]<<' ';
38     cout<<endl;
39 }

```

4.2 Dijkstra

Listing 18: Dijkstra(稀疏图堆优化版本)

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  #define endl '\n'
4  typedef long long ll;
5  const int N=2e5+5;
6  struct node{
7      int u;ll w;
8      friend bool operator<(node a,node b){
9          return a.w>b.w;
10     }
11 };
12 vector<node>G[N];
13 bool vis[N];
14 ll dis[N];
15 int n,m,s;

```

```

16 void Dijkstra(int s){
17     fill(dis+1,dis+1+n,INT_MAX);
18     priority_queue<node>Q;
19     dis[s]=0;
20     Q.push({s,0});
21     while(!Q.empty()){
22         int u=Q.top().u;
23         Q.pop();
24         if(vis[u]) continue;
25         vis[u]=1;
26         for(auto &[v,w]:G[u]){
27             if(dis[v]>dis[u]+w){
28                 dis[v]=dis[u]+w;
29                 Q.push({v,dis[v]});
30             }
31         }
32     }
33 }
34 void solve(){
35     cin>>n>>m>>s;
36     for(int i=1;i<=m;i++){
37         int x,y;ll w;
38         cin>>x>>y>>w;
39         G[x].push_back({y,w});
40     }
41     Dijkstra(s);
42     for(int i=1;i<=n;i++) cout<<dis[i]<<' ';
43     cout<<endl;
44 }
45 int main(){
46     solve();
47     return 0;
48 }

```

Listing 19: Dijkstra(稠密图邻接矩阵版)

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 typedef long long ll;
5 const int N=1010; // 注意:n<=1000 推荐使用稠密版，否则改用稀疏图堆优化！

```

```
6  const ll INF = 1e18;
7
8  ll g[N][N];
9  ll dis[N];
10 bool vis[N];
11 int n,m,s;
12
13 void Dijkstra(int s){
14     for(int i=1;i<=n;i++){
15         dis[i]=INF;
16         vis[i]=false;
17     }
18     dis[s]=0;
19     for(int i=1;i<=n;i++){
20         int u=-1; ll minv=INF;
21         for(int j=1;j<=n;j++){
22             if(!vis[j] && dis[j]<minv){
23                 minv=dis[j];
24                 u=j;
25             }
26         }
27         if(u==-1) break;
28         vis[u]=true;
29         for(int j=1;j<=n;j++){
30             if(g[u][j]<INF && dis[j]>dis[u]+g[u][j]){
31                 dis[j]=dis[u]+g[u][j];
32             }
33         }
34     }
35 }
36
37 void solve(){
38     cin>>n>>m>>s;
39     for(int i=1;i<=n;i++)
40         for(int j=1;j<=n;j++)
41             g[i][j]=(i==j?0:INF);
42     for(int i=1;i<=m;i++){
43         int x,y;ll w;
44         cin>>x>>y>>w;
45         g[x][y]=min(g[x][y],w);    // 支持重边取最优
```

```
46     }
47     Dijkstra(s);
48     for(int i=1;i<=n;i++) cout<<dis[i]<<' ';
49     cout<<endl;
50 }
51 int main(){
52     solve();
53     return 0;
54 }
```

4.3 最小生成树

Listing 20: Kruskal 求最小生成树

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  #define endl '\n'
4  typedef long long ll;
5  ll ans;int n,m,cntn;
6  const int N=2e5+10;
7  int f[N];
8  int find(int x){
9      if(x==f[x]) return x;
10     return f[x]=find(f[x]);
11 }
12 struct node{
13     int x,y,w;
14     friend bool operator <(node a,node b){
15         return a.w<b.w;
16     }
17 }a[N];
18 void kruskal(){
19     cntn=n;
20     for(int i=1;i<=m;i++){
21         int fx=find(a[i].x);
22         int fy=find(a[i].y);
23         if(fx!=fy){
24             f[fx]=fy;
25             cntn--;
26             ans+=a[i].w;
```

```

27         if(cntn==1) break;
28     }
29 }
30 }
31 int main(){
32     cin.tie(0)->ios::sync_with_stdio(false);
33     cin>>n>>m;
34     for(int i=1;i<=n;i++) f[i]=i;
35     for(int i=1;i<=m;i++){
36         int x,y,w;
37         cin>>a[i].x>>a[i].y>>a[i].w;
38     }
39     sort(a+1,a+1+m);
40     kruskal();
41     if(cntn==1) cout<<ans<<endl;
42     else cout<<"none"<<endl;
43     return 0;
44 }

```

4.4 Kruskal 重构树

Kruskal 重构树上任意两点 lca 权值代表两点间最大边权最小值

Listing 21: Kruskal 重构树

```

1 struct DSU{
2     vector<ll>f;
3     DSU(){
4     DSU(int n){
5         f.resize(n+1);
6         iota(f.begin(),f.end(),0);
7     };
8     int find(int x){
9         return x==f[x]?x:f[x]=find(f[x]);
10    }
11    bool merge(int x,int y){
12        int fx=find(x),fy=find(y);
13        if(fx==fy) return false;
14        f[fx]=fy;
15        return true;
16    }

```

```

17 };
18 struct Edge{
19     int x,y,ll w;
20     Edge(){};
21     Edge(int x,int y,ll w):x(x),y(y),w(w){};
22     friend bool operator<(Edge A,Edge B){
23         return A.w<B.w;
24     }
25 };
26 inline void solve(){
27     int n,m;
28     cin>>n>>m;
29     DSU dsu(n<<1);
30     vector<vector<int>>>G(n<<1|1);
31     vector<Edge>a;
32     vector<ll>val(n<<1|1); //存新节点(边)权值
33     for(int i=1;i<=m;i++){
34         int x,y,ll w;
35         cin>>x>>y>>w;
36         a.emplace_back(x,y,w);
37     }
38     sort(a.begin(),a.end());
39     int tot=n;
40     for(auto &[x,y,w]:a){
41         int fx=dsu.find(x),fy=dsu.find(y);
42         if(fx!=fy){
43             ++tot;
44             dsu.merge(fx,tot);
45             dsu.merge(fy,tot);
46             G[tot].push_back(fx);
47             G[tot].push_back(fy);
48             val[tot]=w;
49         }
50     }
51     vector<int>dfn(n<<1|1);
52     vector<ll>d(n<<1|1);
53     vector<vector<int>>>st(n<<1|1,vector<int>(20));
54     int top=0;
55     auto dfs=[&](auto &&dfs,int u,int fa) ->void {
56         d[u]=d[fa]+1;

```

```

57     dfn[u]=++top;
58     st[dfn[u]][0]=fa;
59     for(int v:G[u]) dfs(dfs,v,u);
60 };
61 for(int i=1;i<=(n<<1);i++){
62     if(dsu.find(i)==i){
63         dfs(dfs,i,0);
64     }
65 }
66 for(int i=1;i<20;i++){
67     for(int j=1;j+(1<<i)-1<=(n<<1);j++){
68         int L=st[j][i-1];
69         int R=st[j+(1<<i-1)][i-1];
70         st[j][i]=(d[L]<d[R])?L:R;
71     }
72 }
73 auto lca=[&](int l,int r) ->int {
74     if(l==r) return l;
75     l=dfn[l],r=dfn[r];
76     if(l>r) swap(l,r);
77     ++l;
78     int len=__lg(r-l+1);
79     int L=st[l][len];
80     int R=st[r-(1<<len)+1][len];
81     return (d[L]<d[R])?L:R;
82 };
83 int q;cin>>q;
84 for(int i=1;i<=q;i++){
85     int x,y;cin>>x>>y;
86     if(dsu.find(x)!=dsu.find(y)) cout<<"impossible"<<endl;
87     else cout<<val[lca(x,y)]<<endl;
88 }
89 }

```

4.5 差分约束

给定 n 个变量 $x_1, x_2, \dots, x_n$ 与 m 个差分约束 $x_c - x_{c'} \leq y$ 。

求任意一组满足所有约束的可行解。无解输出 NO。

Listing 22: spfa 差分约束

```
1  const int N=2e5+5;
2  struct node{
3      int u,ll w;
4  };
5  vector<node>G[N];
6  int cnt[N];
7  bool inq[N];
8  ll dis[N];
9  int n,m;
10 bool spfa(){
11     fill(dis+1,dis+1+n,INT_MAX);
12     queue<int>Q;
13     Q.push(0);
14     inq[0]=0;
15     cnt[0]=1;
16     while(!Q.empty()){
17         int u=Q.front();
18         Q.pop();
19         inq[u]=false;
20         for(auto &[v,w]:G[u]){
21             if(dis[v]>dis[u]+w){
22                 dis[v]=dis[u]+w;
23                 if(!inq[v]){
24                     cnt[v]++;
25                     if(cnt[v]>n) return false;
26                     Q.push(v);
27                     inq[v]=1;
28                 }
29             }
30         }
31     }
32     return true;
33 }
34 void solve(){
35     cin>>n>>m;
36     for(int i=1;i<=m;i++){
37         int x,y,ll w;cin>>x>>y>>w;
38         G[y].push_back({x,w});
39     }
```

```
40     for(int i=1;i<=n;i++){  
41         G[0].push_back({i,0});  
42     }  
43     if(spfa()){  
44         for(int i=1;i<=n;i++) cout<<dis[i]<<' '  
45         cout<<endl;  
46     }  
47     else cout<<"NO"<<endl;  
48 }
```

4.6 缩点

5 字符串

5.1 字符串哈希

Listing 23: 字符串哈希

```
1 using ull = unsigned long long;
2 ull base = 131;
3 ull mod1 = 212370440130137957, mod2 = 1e9 + 7;
4 ull get_hash1(const string &s) {
5     int len = s.size();
6     ull ans = 0;
7     for (int i = 0; i < len; i++) ans = (ans * base + (ull)s[i]) % mod1;
8     return ans;
9 }
10 ull get_hash2(const string &s) {
11     int len = s.size();
12     ull ans = 0;
13     for (int i = 0; i < len; i++) ans = (ans * base + (ull)s[i]) % mod2;
14     return ans;
15 }
16 bool cmp(const string &s, const string &t) {
17     bool f1 = get_hash1(s) != get_hash1(t);
18     bool f2 = get_hash2(s) != get_hash2(t);
19     return f1 || f2;
20 }
```

5.2 KMP

5.3 后缀树

5.4 马拉车

6 动态规划

6.1 01 背包

6.2 完全背包

6.3 多重背包

6.4 树形 dp

6.5 换根 dp

6.6 状压 dp

6.7 SOS dp

6.8 数位 dp

7 数学

7.1 筛法求因数个数

Listing 24: 因数筛

```

1  const int N = 1000010;
2  int p[N];    // p[i]: i 的最小质因子 (spf)
3  int pr[N];   // pr[]: 质数表
4  int cnt;
5  int d[N];    // d[i]: i 的正因数个数
6  int a[N];    // a[i]: i 的最小质因子的指数
7  void init(int n) {
8      d[1] = 1; a[1] = 0; p[1] = 1;
9      for(int i = 2; i <= n; i++) {
10         if(!p[i]) {                // i 是质数
11             p[i] = pr[++cnt] = i;
12             d[i] = 2;
13             a[i] = 1;
14         }
15         for(int j = 1; j <= cnt && 1LL * i * pr[j] <= n; j++) {
16             int m = i * pr[j];
17             p[m] = pr[j];           // 记录 m 的最小质因子
18             if(p[i] == pr[j]) {    // 还是同一个质因子
19                 d[m] = d[i] / (a[i] + 1) * (a[i] + 2);
20                 a[m] = a[i] + 1;
21                 break;
22             }
23             else {                  // 新增一个不同质因子
24                 d[m] = d[i] * 2;    // 等价于 d[i] * d[pr[j]]
25                 a[m] = 1;
26             }
27         }
28     }
29 }

```

7.2 筛法求欧拉函数

Listing 25: 欧拉筛

```

1  const int N=1e6+5;

```

```
2 int p[N],vis[N],cnt;
3 int phi[N];
4 void init(){
5     phi[1]=1;
6     for(int i=2;i<=n;i++){
7         if(!vis[i]){
8             p[cnt++]=i;
9             phi[i]=i-1;
10        }
11        for(int j=0;i*p[j]<=n;j++){
12            int m=i*p[j];
13            vis[m]=1;
14            if(i%p[j]==0){
15                phi[m]=p[j]*phi[i];
16                break;
17            }
18            else phi[m]=(p[j]-1)*phi[i];
19        }
20    }
21 }
```

7.3 筛法求莫比乌斯函数

Listing 26: 莫比乌斯函数筛

```
1  const int N=1e6+5;
2  int p[N],vis[N],cnt;
3  int mu[N];
4  void init(){
5      mu[1]=1;
6      for(int i=2;i<=n;i++){
7          if(!vis[i]){
8              p[cnt++]=i;
9              mu[i]=-1;
10         }
11         for(int j=0;i*p[j]<=n;j++){
12             int m=i*p[j];
13             vis[m]=1;
14             if(i%p[j]==0){
15                 mu[m]=0;
16                 break;
17             }
18             else{
19                 mu[m]=-mu[i];
20             }
21         }
22     }
23 }
```

7.4 矩阵快速幂

Listing 27: 矩阵快速幂 basic

```

1 struct Mat{
2     vector<vector<ll>>>a;
3     int n;
4     Mat(int n):n(n){
5         a=vector<vector<ll>>>(n+1,vector<ll>(n+1));
6     };
7     Mat(){};
8     friend Mat operator+(Mat &A,Mat &B){
9         Mat C(A.n);
10        for(int i=0;i<=A.n;i++){
11            for(int j=0;j<=A.n;j++){
12                C.a[i][j]=(A.a[i][j]+B.a[i][j])%mod;
13            }
14        }
15        return C;
16    }
17    friend Mat operator+=(Mat &A,Mat &B){
18        A=A+B;
19        return A;
20    }
21    friend Mat operator*(Mat &A,Mat &B){
22        Mat C(A.n);
23        for(int k=0;k<=A.n;k++){
24            for(int i=0;i<=A.n;i++){
25                if(A.a[i][k]==0) continue;
26                for(int j=0;j<=A.n;j++){
27                    C.a[i][j]=(C.a[i][j]+A.a[i][k]*B.a[k][j]%mod)%mod;
28                }
29            }
30        }
31        return C;
32    }
33    friend Mat operator*=(Mat &A,Mat &B){
34        A=A*B;
35        return A;
36    }
37    void q_pow(Mat &A,ll B){

```



```

38     Mat S(A.n); //单位矩阵
39     for(int i=0; i<=A.n; i++) S.a[i][i]=1;
40     while(B>0){
41         if(B&1){
42             S*=A;
43         }
44         A*=A;
45         B>>=1;
46     }
47     A=move(S);
48 }
49 }M;

```

Listing 28: 矩阵快速幂 max-plus

```

1  const ll INF=1e18;
2  struct Mat{
3      vector<vector<ll>>>a;
4      int n;
5      Mat(int n):n(n){
6          a=vector<vector<ll>>>(n+1,vector<ll>(n+1,-INF));
7      };
8      Mat(){};
9      friend Mat operator*(const Mat &A,const Mat &B){
10         Mat C(A.n);
11         for(int k=0;k<=A.n;k++){
12             for(int i=0;i<=A.n;i++){
13                 for(int j=0;j<=A.n;j++){
14                     C.a[i][j]=max(C.a[i][j],A.a[i][k]+B.a[k][j]);
15                 }
16             }
17         }
18         return C;
19     }
20     friend Mat operator*=(Mat &A,Mat &B){
21         A=A*B;
22         return A;
23     }
24     void q_pow(Mat &A,ll B){
25         Mat S(A.n); //单位矩阵
26         for(int i=0;i<=A.n;i++) S.a[i][i]=0;

```

```

27     while(B>0){
28         if(B&1) S*=A;
29         A*=A;
30         B>>=1;
31     }
32     A=move(S);
33 }
34 }M;

```

7.5 第二类斯特林数

第二类斯特林数（斯特林子集数） $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ ，也可记做 $S(n, k)$ ，表示将 n 个两两不同的元素划分为 k 个互不区分的非空子集的方案数。

Listing 29: Stirling 数

```

1  typedef unsigned long long ll;
2  const int N=10010;
3  ll dp[N][N/10],p;
4  void init(){
5      dp[1][1]=1;
6      for(int i=2;i<=10000;i++){
7          for(int j=1;j<=1000;j++){
8              dp[i][j]=((j%p)*dp[i-1][j]%p+dp[i-1][j-1])%p;
9          }
10     }
11 }
12 int main(){
13     cin.tie(0)->ios::sync_with_stdio(false);
14     int n,m;
15     cin>>n>>m>>p;
16     init();
17     cout<<dp[n][m]<<endl;
18     return 0;
19 }

```

7.6 卡特兰数

Catalan 数有如下常见的表达式:

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{n!(n+1)!}, \quad n \geq 0. \quad (2)$$

$$C_n = \binom{2n}{n} - \binom{2n}{n+1}, \quad n \geq 0. \quad (3)$$

$$C_n = \frac{4n-2}{n+1} C_{n-1}, \quad n > 0, \quad C_0 = 1. \quad (4)$$

数列的前几项为: 1,1,2,5,14,42,132,429,1430,...

7.7 排列组合类-含 Lucas

7.8 高斯消元

7.9 线性基

7.10 数论分块

7.11 莫比乌斯反演

7.12 exgcd

7.13 位运算基础